

Softver za upravljanje voznim parkom

Razvoj MVVM WPF aplikacije uz upotrebu Entity Framework Core i UML modelovanja

Datum: 02/02/2026

Projektna dokumentacija

1. Uvod

Softver za upravljanje voznim parkom je desktop aplikacija razvijena koristeći **Windows Presentation Foundation (WPF)** na **.NET 6/8 platformi**. Aplikacija omogućava administratorima, menadžerima i vozačima da efikasno upravljaju voznim parkom kroz dodavanje i ažuriranje vozila, pregled i filtriranje podataka, autentifikaciju korisnika, serijalizaciju informacija i generisanje detaljnih PDF izveštaja.

1.1. Cilj Projekta

- Cilj ovog projekta je razvoj potpuno funkcionalne desktop aplikacije koja demonstrira:
- Implementaciju **MVVM arhitekturnog obrasca** radi jasne separacije slojeva
- Korišćenje **Entity Framework Core** za rad sa bazom podataka (SQLite)
- Primenu dizajn šablona (**Singleton, Factory, Observer**) u ključnim delovima aplikacije
- **Serijalizaciju podataka** u JSON i XML formatima radi arhiviranja i razmene
- **Generisanje PDF izveštaja** o voznom parku pomoću PdfSharp biblioteke
- **Implementaciju autentifikacije korisnika** sa različitim ulogama (administrator, menadžer, vozač)
- **Jedinično testiranje kritičnih komponenti** aplikacije korišćenjem xUnit i EF Core InMemory providera

1.2 Tehnologije i alati

Kategorija	Tehnologija / Alat
UI	WPF (.NET 6+), XAML
Arhitektura	MVVM (Model-View-ViewModel)
ORM	Entity Framework Core
Baza podataka	SQLite
Verzionsanje	Git, GitHub
Testiranje	xUnit
Serijalizacija	JSON (System.Text.Json), XML (XmlSerializer)
Dokumentacija	draw.io

Analiza

Glavni Use Case-ovi:

- 1. Dodavanje novog vozila**
- 2. Izmena i brisanje vozila**
- 3. Autentifikacija (login)**
- 4. Pregled i filtriranje vozila**
- 5. Pregled sopstvenih vozila**
- 6. Generisanje izveštaja (PDF)**
- 7. Serijalizacija podataka (JSON/XML)**

2. Modelovanje

2.1 Opis Use Case dijagrama

Akteri:

- **Administrator** – dodaje, menja i briše vozila, serijalizuje podatke, generiše izveštaje.
- **Menadžer** – koristi sistem za pregled i filtriranje podataka, kao i za kreiranje izveštaja.
- **Vozač** – ima mogućnost pregleda sopstvenih vozila i servisne istorije.

Use Case-ovi:

1. **Dodavanje novog vozila** – Administrator unosi podatke o novom vozilu (marka, model, registracija, status).
2. **Izmena i brisanje vozila** – Administrator ažurira podatke ili uklanja vozilo iz sistema.
3. **Autentifikacija (login)** – Svi korisnici se prijavljuju u sistem putem login forme.
4. **Pregled i filtriranje vozila** – Menadžer pregleda i filtrira sva vozila po statusu, tipu ili datumu servisa. Vozač ima mogućnost pregleda samo vozila koja su njemu dodeljena.
5. **Pregled sopstvenih vozila** – Vozač pregleda vozila koja su njemu dodeljena.
6. **Generisanje izveštaja (PDF)** – Menadžer kreira PDF izveštaj o voznom parku (lista vozila, statusi, servisni podaci).
7. **Serijalizacija podataka (JSON/XML)** – Administrator eksportuje podatke radi arhiviranja ili razmene.

Relacije:

- **Administrator** je povezan sa: Dodavanje vozila, Izmena/brisanje, Serijalizacija podataka.
- **Menadžer** je povezan sa: Pregled i filtriranje, Generisanje izveštaja.
- **Vozač** je povezan sa: Pregled sopstvenih vozila.
- **Svi aktori** su povezani sa: Autentifikacija

2.2 Dijagram Klasa

1. Apstraktna klasa:

- Osoba

2. Nasleđene klase

- Vozač
- Administrator
- Menadžer

3. Entiteti

- Vozilo
- Servis

4. Interfejs

- IReportGenerator

5. Relacije

- Vozač, Administrator, Menadžer nasleđuju Osoba
- Vozilo ima kompoziciju sa Servis ($1 \rightarrow *$)
- Vozač je u asocijaciji sa Vozilo ($1 \rightarrow 0..*$)
- Menadžer koristi podatke o vozilima i vozačima
- ReportService implementira IReportGenerator

6. Multipliciteti

- Vozilo $\rightarrow$ Servis: $1 \rightarrow *$
- Vozač $\rightarrow$ Vozilo: $1 \rightarrow *$

2.3 Dijagram paketa

- **Korisnici**
 - Klase: Osoba, Vozač, Administrator, Menadžer
 - Predstavlja korisničke uloge u sistemu
- **VozniPark**
 - Klase: Vozilo, Servis
 - Obuhvata entitete vezane za vozni park i servisnu istoriju
- **Izveštaji**
 - Interfejs i klasa: IReportGenerator, ReportService
 - Omogućava generisanje izveštaja na osnovu podataka iz sistema

3. Implementacija

3.1 Arhitektura

Aplikacija je razvijena po MVVM (Model–View–ViewModel) arhitektonskom obrascu, što omogućava jasnu separaciju logike, podataka i korisničkog interfejsa:

- **Model** – obuhvata entitete Vozilo, Servis i apstraktnu klasu Osoba sa izvedenim klasama (Administrator, Menadžer, Vozač). Model definiše strukturu podataka i relacije između njih, kao i mapiranje ka bazi putem EF Core.
- **ViewModel** – sadrži poslovnu logiku aplikacije, validaciju i binding prema View sloju. Implementira ICommand interfejs za rukovanje akcijama korisnika (npr. dodavanje vozila, login). ViewModel je odgovoran za komunikaciju sa Model slojem i za pripremu podataka za prikaz.
- **View** – XAML forme (LoginView, VozilaView, ServisiView, IzveštajiView) koje predstavljaju korisnički interfejs. One koriste data binding da se povežu sa ViewModelima, čime se eliminiše potreba za logikom u code-behind fajlovima.

3.2 Entity Framework Core

Za rad sa bazom podataka koristi se Entity Framework Core:

- **Baza:** SQLite
- **Entiteti:** Vozilo, Servis, User (sa izvedenim klasama).
- **CRUD operacije:** implementirane kroz ViewModele i DbContext. Korisnik može dodavati nova vozila, menjati podatke, brisati vozila ili pregledati listu postojećih. Odradjen je CRUD za servise takodje.

3.3 Serijalizacija

Podaci se mogu eksportovati radi arhiviranja ili razmene:

- **JSON (System.Text.Json):** koristi se za brzu i modernu serijalizaciju podataka.

3.4 Izveštaji

- Izveštaj se kreira u PDF formatu i sadrži listu vozila, njihovo stanje, servisne podatke i filtrirane informacije.
- Menadžer može da generiše izveštaj u bilo kom trenutku, što omogućava pregled stanja voznog parka u standardizovanom formatu pogodnom za štampu ili deljenje.

3.5 Dizajn šabloni

- **Factory** – Korišćen za kreiranje različitih tipova korisnika (Administrator, Menadžer, Vozač) na osnovu apstraktne klase Osoba. Factory šablon omogućava centralizovano i fleksibilno instanciranje objekata, čime se olakšava proširivost sistema.
- **Observer** – Implementiran kroz INotifyPropertyChanged interfejs u ViewModelima. Kada se vrednost u Modelu ili ViewModelu promeni, View automatski reaguje i ažurira prikaz. Ovo obezbeđuje dinamičan i respozivan korisnički interfejs.

3.6 Izveštaji:

- Izveštaj se kreira pomoću biblioteke **PdfSharp**, koja omogućava precizno formatiranje teksta, tabela i grafičkih elemenata.
- Sadržaj izveštaja obuhvata:
 - Listu svih vozila u sistemu (marka, model, registracija).
 - Status vozila (aktivno, u servisu, rashodovano).
 - Servisne podatke (datum poslednjeg servisa, opis intervencije).
 - Filtrirane informacije na osnovu kriterijuma koje menadžer odabere (npr. samo vozila u servisu).
- Izveštaj se automatski generiše u PDF formatu i može se sačuvati lokalno ili proslediti drugim korisnicima.

3.7 Slike aplikacije:

Pocetni prozor:

Fleet Management System

 Pregled Vozila

 Pregled Servisa

 Izveštaji

Pregled Vozila

Pregled vozila

Pregled svih vozila

 Pretraga vozila:

Marka	Model	Godina	Registracija	Status
Dacia	Logan	2006	CU-003-NS	Aktivno
Dacia	Logan	2006	CU-003-NS	Aktivno
Mercedes	S klasa	2020	CU-001-BG	Aktivno
Novo vozilo	Novo	2010	KG-002-CU	aktivno
Novo vozilo	Mfekomfkelw	2020	rfgmeflkwe	42eaaa

+ Dodaj

 Izmeni

 Obriši

Dodavanje vozila:

Marka:	Reno
Model:	Clio
Godina proizvodnje:	2009
Registracija:	KG-002-NS
Status:	Aktivno

Sačuvaj

Otkazi

Izmena vozila:

Izmeni Vozilo

Marka:	Reno
Model:	Clio
Godina:	2009
Registracija:	KG-002-NS
Status:	Aktivno

Sačuvaj

Otkazi

Pregled servisa

Pregled Servisa

Pregled svih servisa

Pretraga servisa:

Opis	Datum	Vozilo ID
Servis 1	12/28/2025	2
Servis 2	2/1/2026	6
Servis 3	2/2/2026	3

Dodaj

Izmeni

Obrisi

Izveštaj Servisi

Broj Servisa

Export

Import

Eksport u PDF I JSON:

Izveštaji

Generisanje izveštaja

Servisi po vozilu (PDF)

Broj servisa po vozilu (PDF)

Export JSON

4. Testiranje

Testiranje aplikacije je realizovano kroz poseban projekat **FleetManagement.Tests**, koristeći **xUnit** framework i **Entity Framework Core InMemory provider**. Na ovaj način obezbeđeno je da se poslovna logika aplikacije proverava izolovano, bez potrebe za stvarnom bazom podataka.

Ciljevi testiranja:

- Validacija osnovnih funkcionalnosti ViewModel sloja.
- Provera ispravnosti CRUD operacija nad entitetima.
- Testiranje generisanja izveštaja i serijalizacije podataka.
- Obezbeđivanje stabilnosti aplikacije kroz automatizovane testove.

5. Git i Verzionisanje

Projekat koristi Git za verzionisanje koda i GitHub za hosting repozitorijuma. Verizonisanje koda, vršilo se na main grani, dok je radi demonstracije “Pull Request-a” bila dodata nova grana za login i register koja je posle mergovana sa main granom.

5.1 Commit istorija

1. Inicijalni komit
2. Dodata logika za modele i relacije
3. Dodat DbContext i inicijalna migracija
4. Dodati fajlovi BaseViewModel i RelayCommand
5. Dodati BaseViewModel i RelayCommand fajlovi
6. Dodata forma za unos Vozilo i logika za dodavanje vozila
7. Implementiran CRUD servisa
8. Dodata funkcionalnost za serijalizaciju i eksport u pdf fajlove
9. Dodata funkcionalnost za filtriranje vozila i servisa
10. Implementiran login/register
11. Merge pull request #1 from lanchemi11/login-register-feature
12. Dodati primeri za Factory i Observer dizajn sablone
13. Dodato testiranje
14. Dodati testovi

15. Create README.md

16. Dodati Dijagrami

17. Quick Fix

Grana koja se mergovala: login-register-feature

6. Zaključak

Projekat **Softver za upravljanje voznim parkom** uspešno demonstrira primenu savremenih tehnologija i arhitekturnih obrazaca u razvoju desktop aplikacija. Kroz implementaciju **MVVM** strukture, korišćenje **Entity Framework Core** za rad sa bazom podataka, primenu dizajn šablona (**Factory, Observer, Singleton, Command**) i dodatne funkcionalnosti poput serijalizacije i generisanja PDF izveštaja, aplikacija pruža sveobuhvatno rešenje za upravljanje voznim parkom.

Razvoj je obuhvatio sve ključne faze, od analize i modelovanja, preko implementacije i testiranja, do dokumentovanja i verzionisanja koda. Time je obezbeđena ne samo funkcionalnost, već i održivost i proširivost sistema.

Konačan rezultat je aplikacija koja omogućava administratorima, menadžerima i vozačima da efikasno obavljaju svoje zadatke, uz intuitivan korisnički interfejs i pouzdanu logiku u pozadini. Projekat pokazuje sposobnost integracije teorijskih znanja (UML modelovanje, dizajn šabloni) sa praktičnom implementacijom u realnom softverskom okruženju.